Final FCIT-20

▼ Question 1

In question number 1, you are given two code snippets that involve a function `fun()`, and you are asked to provide the output for each snippet. Below is an example of a simple `fun()` function in C:

Let's analyze the provided code and understand why the values of `b1` and `b2` are 40 and 35, respectively.

```
int a = 10;

int fun(int x) {
    a = 5;
    return 20;
}

int main() {
    // Code snippet 1: b = a + 10 + fun(a);
    int b1 = a + 10 + fun(a);

    // Code snippet 2: b = fun(a) + 10 + a;
    int b2 = fun(a) + 10 + a;

    // Print the values of b1 and b2
    printf("Value of b1: %d\\n", b1);
    printf("Value of b2: %d\\n", b2);

    return 0;
}
```

Explanation:

Code Snippet 1 (`b1 = a + 10 + fun(a);`):

1. Initial value of `a` is 10.

2. `fun(a)` is called. Inside `fun()`, the value of `a` is changed to 5, but this doesn't affect the calculation of `b1` since it's passed by value.
3. The calculation becomes `b1 = 10 + 10 + 20 = 40`.

Code Snippet 2 (`b2 = fun(a) + 10 + a;`):

1. `fun(a)` is called again. Inside `fun()`, the value of `a` is changed to 5.
2. The calculation becomes `b2 = 20 + 10 + 5 = 35`.

In summary:

- In the first case, the change in the value of `a` inside `fun()` has no effect on the calculation of `b1` because `a` is passed by value to the function.
- In the second case, the change in the value of `a` inside `fun()` affects the calculation of `b2` because `a` is modified globally.

So, the output will be:

```
Value of b1: 40
Value of b2: 35
```

▼ Question 2

Comparing the loose equality (`==`) and strict equality (`===`) operators.

PHP Example:

```
<?php

// Code snippet 1: Loose equality comparison
echo "7" == 7; // Output: 1 (true)

// Code snippet 2: Strict equality comparison
echo "7" === 7; // Output: (empty)

?>
```

Explanation (PHP):

Code Snippet 1 (`"7" == 7`):

- The loose equality (`==`) operator in PHP checks for equality after performing type coercion.
- In this case, the string "7" is automatically converted to the integer 7, and the comparison becomes `7 == 7`.
- The output is `1`, indicating true.

Code Snippet 2 (`"7" === 7`):

- The strict equality (`===`) operator in PHP checks for equality without performing type coercion.
- In this case, the string "7" is not equal to the integer 7 in both value and type, so the comparison is false.
- The output is an empty value.

JavaScript Example:

```
// Code snippet 1: Loose equality comparison
console.log("7" == 7); // Output: true

// Code snippet 2: Strict equality comparison
console.log("7" === 7); // Output: false
```

Explanation (JavaScript):

Code Snippet 1 (`"7" == 7`):

- The loose equality (`==`) operator in JavaScript performs type coercion, so the string "7" is coerced to the number 7.
- The comparison becomes `7 == 7`, and the output is `true`.

Code Snippet 2 (`"7" === 7`):

- The strict equality (`===`) operator in JavaScript checks for both value and type.
- The string "7" is not equal to the number 7 in type, so the comparison is false.

- The output is `false`.

In both PHP and JavaScript, the loose equality (`==`) performs type coercion, while the strict equality (`===`) does not. This can lead to different results when comparing values of different types.

Let's go through examples in both Python and C.

Python Example:

```
# Code snippet 1: Loose equality comparison
print("7" == 7)  # Output: False

# Code snippet 2: Strict equality comparison
print("7" === 7)  # This line will result in a syntax error in Python
```

Explanation (Python):

- Python does not have a loose equality operator like PHP or JavaScript.
- The attempt to use `===` in Python will result in a syntax error because Python uses `==` for both loose and strict equality comparisons.
- Comparing a string and an integer using `==` will result in `False` because the types are different.

C Example:

```
#include <stdio.h>

int main() {
    // Code snippet 1: Loose equality comparison
    printf("%d\\n", "7" == 7);  // Output: 1 (true)

    // Code snippet 2: Strict equality comparison
    printf("%d\\n", "7" === 7);  // This line will result in a compilation error in C

    return 0;
}
```

Explanation (C):

- C does not have built-in string types like Python, so comparing a string literal and an integer directly will result in a comparison based on their numerical values.
- The loose equality (`==`) comparison will be true because both the string literal "7" and the integer 7 have the same numerical value.
- The attempt to use `===` in C will result in a compilation error because C does not have a strict equality operator like JavaScript.

In summary, both Python and C behave differently from PHP and JavaScript in terms of equality comparisons. Python uses `==` for both loose and strict equality, and C lacks native string types and strict equality operators.

▼ Question 3

C code:

```
int i = 0;
int j = 0;
for (i = 0, j = 0; j <= 5 || i > 3; j++, i += 1) {
    if (j == 3) continue;
    if (i == 4) break;
    j += 2;
}
```

Operational Semantic Description for C “For” Statement:

```
i = 0
j = 0
loop:
  if j > 5 goto out
  if i <= 3 goto out
  if j == 3:
    goto loop
  if i == 4:
    goto out
  j = j + 2
  j = j + 1
  i = i + 1
  goto loop
out:...
```

Explanation:

1. **Initialization:**
   - `i` is set to 0.
   - `j` is set to 0.
2. **Loop Condition Check:**
   - If `j` is greater than 5, jump to the "out" label.
   - If `i` is less than or equal to 3, jump to the "out" label.
3. **Additional Conditions:**
   - If `j` is equal to 3, jump back to the "loop" label (using `continue`).
   - If `i` is equal to 4, jump to the "out" label (using `break`).
4. **Loop Body Execution:**
   - Increment `j` by 2.
   - Increment `j` by 1.
   - Increment `i` by 1.
5. **Loop Continuation:**
   - Jump back to the "loop" label to re-evaluate the loop condition.
6. **Loop Termination:**
   - If the loop condition is false, jump to the "out" label.

This description captures the essence of the provided C "for" loop using operational semantics. The "loop" label signifies the beginning of the loop, and the "out" label is where the program goes when the loop terminates.

▼ Question 4

1. Create a language using BNF and improve it using EBNF:

BNF (Backus-Naur Form) Example:

```
<declaration> ::= <type> <variable_list> ;
<type> ::= int | float | double
```

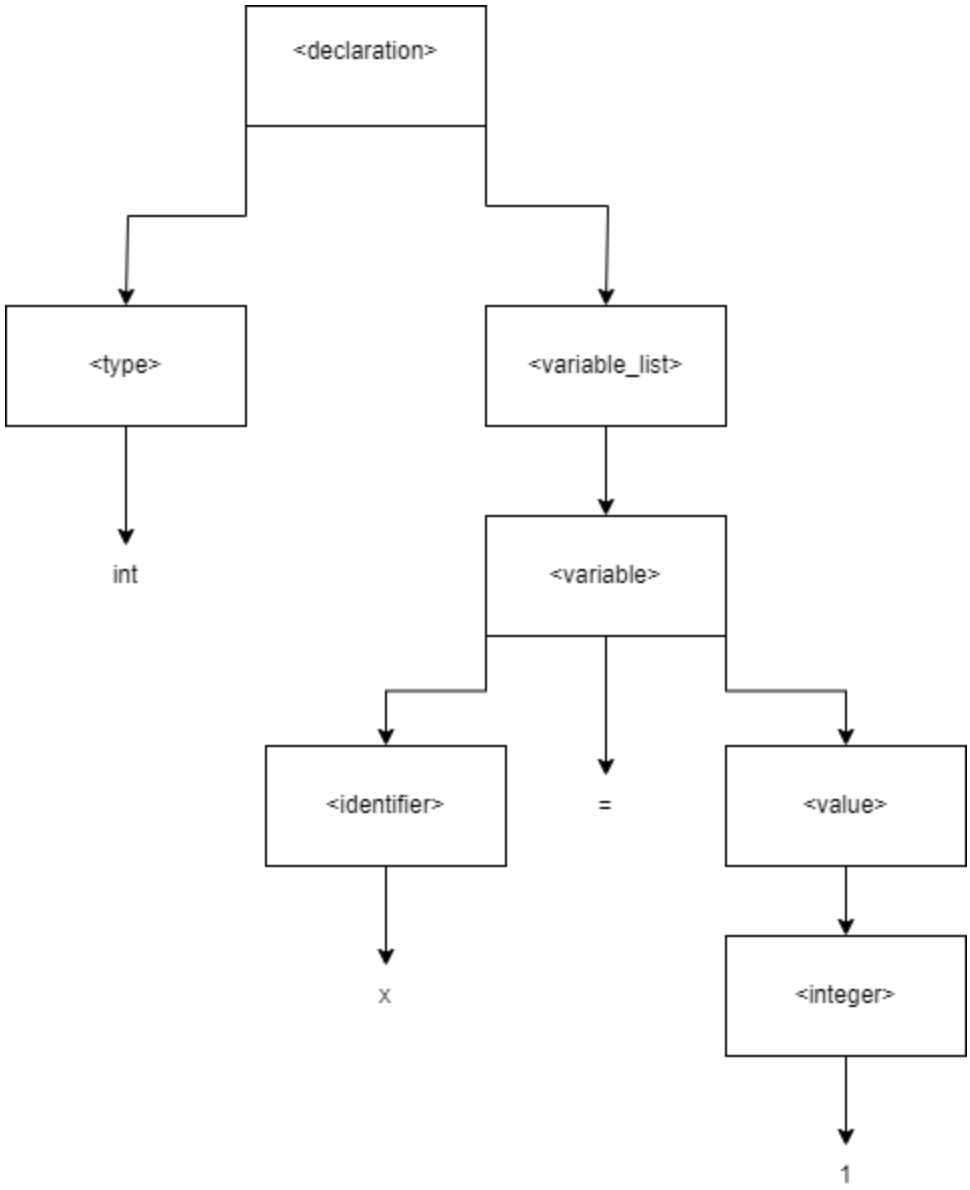
```
<variable_list> ::= <variable> | <variable> , <variable_list>
<variable> ::= <identifier> = <value> | <identifier>
<value> ::= <integer> | <float_literal>
<integer> ::= [0-9]+
<float_literal> ::= [0-9]+.[0-9]+
<identifier> ::= [a-zA-Z][a-zA-Z0-9]*
```

EBNF (Extended Backus-Naur Form) Improved Example:

```
<declaration> ::= <type> <variable_list> ;
<type> ::= int | float | double
<variable_list> ::= <variable> {, <variable>}*
<variable> ::= <identifier> = <value> | identifier
<value> ::= <integer>| <float_literal>
<integer> ::= [0-9]+
<float_literal> ::= [0-9]+.[0-9]+
<identifier> ::= [a-zA-Z][a-zA-Z0-9]*
```

2. Determine if the grammar is ambiguous and provide examples:

The provided grammar is **not ambiguous**. Each non-terminal has a clear and unambiguous definition. Ambiguity in a grammar would arise if there were multiple interpretations for the same input.



3. Examples of Accepted and Rejected Statements:

Accepted Statements:

```
int x = 0;
int x, y;
float x;
double t;
int y = 1;
```

Rejected Statements:

```
Int x, int y;  // Case-sensitive, should be "int"
X int;          // "X" is not a valid type
flouty;         // "flouty" is not a valid type
```